Universidad Católica Andrés Bello
Facultad de Ingeniería
Escuela de Ingeniería Informática
Tópicos Especiales de Programación

LogiExpress: motor analítico de vuelos e itinerarios interurbanos

Informe Técnico Comparativo
Paradigma lógico (Prolog) y paradigma funcional (Haskell)

Integrantes:

Maria Rodriguez

Lynda Nasser

Jesus Montoya

Profesora: Jannelly Bello

Noviembre de 2026

1. Introducción

LogiExpress es una empresa de transporte aéreo que requiere un motor analítico para su sistema de gestión de vuelos e itinerarios interurbanos. A partir de un archivo plano `vuelos.txt` con veinte vuelos entre ocho ciudades, el sistema debe determinar todas las rutas válidas entre un origen y un destino para un día de la semana, y seleccionar entre ellas la de menor costo y la de menor tiempo total de viaje.

La red de vuelos es un grafo dirigido y ponderado: cada ciudad es un nodo y cada vuelo una arista con dos pesos, el costo del boleto y el par de horas de salida y llegada. Lo que aleja el problema de una búsqueda de caminos clásica son las restricciones simultáneas sobre cada itinerario: cada vuelo debe operar el día consultado; la salida de cada vuelo debe ser mayor o igual a la llegada del anterior más el Tiempo Mínimo de Conexión (TMC), de modo que importa a qué hora llegamos a cada ciudad y no solo en cuál estamos; ninguna ciudad puede repetirse, y la base de datos contiene ciclos reales como Pto Ordaz ↔ Maiquetia; y, opcionalmente, se fija un máximo de K escalas. El primer reto fue el tiempo: las horas vienen como texto `HH:MM`, sobre el que no se puede sumar ni comparar con seguridad, por lo que las convertimos a minutos desde las 00:00 (07:30 → 450) al cargar el archivo, y toda la aritmética posterior se hace sobre enteros.

El objetivo académico fue resolver el mismo problema con dos formas de pensar distintas: en el paradigma lógico (SWI-Prolog) describimos qué es una ruta válida mediante hechos y reglas y dejamos que el motor de inferencia encuentre las soluciones por unificación y *backtracking*; en el paradigma funcional (Haskell) construimos un grafo inmutable y expresamos la búsqueda como composición de funciones puras sobre listas, apoyándonos en la evaluación perezosa. Ambas implementaciones leen el mismo archivo, ofrecen el mismo menú de consola y, como comprobamos en las pruebas, producen las mismas rutas para todas las combinaciones de origen, destino y día.

2. Explicación del Modelado de Datos (Lógico y Funcional)

2.1 Modelado funcional en Haskell

En Haskell modelamos el dominio con tipos algebraicos de datos (`Tipos.hs`): el día de la semana es un tipo enumerado propio y el vuelo un registro con campos nombrados:

```
data Dia = Lunes | Martes | Miercoles | Jueves | Viernes | Sabado | Domingo
  deriving (Eq, Ord, Show, Enum, Bounded)

data Vuelo = Vuelo
  { codigo :: String, origen :: Ciudad, destino :: Ciudad
  , salida :: Minutos, llegada :: Minutos        -- minutos desde las 00:00
  , costo :: Int, frecuencia :: [Dia] }          -- días en los que opera
  deriving (Eq, Show)

type Ruta  = [Vuelo]
type Grafo = [(Ciudad, [Vuelo])]
```

Las clases derivadas nos dieron comportamientos sin escribir código: `Eq` para comprobar si un vuelo opera cierto día y `Enum` con `Bounded` para expresar la frecuencia `Diario` como `[minBound .. maxBound]`. El grafo es una lista de adyacencia que asocia a cada ciudad los vuelos que parten de ella; se construye una sola vez al cargar el archivo (`Grafo.hs`, con `sortOn` y `groupBy`) y, por ser inmutable, todas las consultas lo comparten sin riesgo de que una búsqueda altere los datos de otra. Para los errores usamos `Maybe` y `Either` en lugar de excepciones: `horaAMinutos` devuelve `Nothing` ante una hora mal formada y `parsearLinea` devuelve en un `Either` el mensaje de la línea rechazada. No existe una variable global para el TMC ni para los vuelos: el estado de la sesión es un registro `Estado` que el bucle del menú recibe como argumento y cada operación devuelve uno nuevo. Toda la entrada y salida quedó confinada en `Main.hs`; los demás módulos son funciones puras.

2.2 Modelado lógico en Prolog

En Prolog no hay tipos declarados: la forma de los datos es el término que elegimos. Cada vuelo es un hecho de aridad siete, con las horas en minutos y los días como lista de átomos:

```
:- dynamic vuelo/7.
%  vuelo(Origen, Destino, Salida, Llegada, Costo, Codigo, Dias)
vuelo('Pto Ordaz', 'Maiquetia', 420, 480, 5600, 'Vlo#1',
      [lunes, martes, miercoles, jueves, viernes, sabado, domingo]).
```

La directiva `:- dynamic vuelo/7` fue indispensable porque los hechos no están en el código fuente: `cargar_vuelos/1` (`carga.pl`) lee el archivo con `read_file_to_string/3`, separa líneas y campos con `split_string/4`, convierte las horas con `hora_a_minutos/2` y agrega cada hecho con `assertz/1`, tras un `retractall/1` que evita duplicados al recargar;

el TMC es otro hecho dinámico, `tmc/1`, que `configurar_tmc/1` reemplaza igual. Aquí está la diferencia conceptual más grande respecto a Haskell: el grafo no es una estructura de datos, es la propia base de conocimiento. Las aristas que salen de una ciudad se obtienen con `vuelo('Pto Ordaz', Destino, S, L, C, V, F)` y las que llegan con `vuelo(Origen, 'Maiquetia', S, L, C, V, F)`, el mismo hecho en ambos sentidos gracias a la unificación, mientras que en Haskell la lista de adyacencia solo indexa por origen y para las llegadas recorrimos el grafo completo. La frecuencia se comprueba con `memberchk(Dia, Dias)`, una ruta es una lista de términos `vuelo/7` y el límite de escalas es un entero o el átomo `sin_limite`.

2.3 Comparación del modelado

Aspecto	Prolog	Haskell
Representación del grafo	Hechos `vuelo/7` en la base de conocimiento	Lista de adyacencia `[(Ciudad, [Vuelo])]`
Tipado	Dinámico; el contrato es la aridad y la posición de los argumentos	Estático; el compilador verifica campos y tipos
Consulta por destino	El mismo hecho con otro argumento instanciado	Recorrido del grafo con una *list comprehension*
Estado (TMC, datos cargados)	Hechos dinámicos modificados con `assertz` y `retractall`	Registro `Estado` inmutable pasado como argumento
Ausencia o error	Fallo del predicado (`false`)	Valores `Maybe` y `Either`

3. Funciones y Predicados Investigados

3.1 Haskell

List comprehensions. Fue la herramienta que más usamos: combina generadores (`x <- lista`), guardas que filtran y una expresión de salida, y varios generadores encadenados equivalen a bucles anidados. La consulta de llegadas a una ciudad recorre el grafo con dos generadores y una guarda: `[ v | (_, vs) <- grafo, v <- vs, destino v == c ]`.

Búsqueda en profundidad sin variables mutables. Lo que más nos costó fue reescribir el DFS que conocíamos (una pila, un conjunto de visitados que se modifica y un bucle `while`) en un lenguaje donde nada se modifica. La solución, en `Rutas.hs`, es una función recursiva cuyo cuerpo es una sola *comprehension*: cada generador es un punto de decisión y cada guarda una de las cuatro restricciones del enunciado.

```
dfs actual visitados anterior tomados
  | actual == hasta = [reverse tomados]          -- caso base: ruta completa
  | otherwise =
      [ ruta
      | v <- vuelosDesde grafo actual               -- aristas salientes
      , disponibleEn dia v                          -- (1) frecuencia
      , destino v `notElem` visitados               -- (3) sin ciclos
      , maybe True (\p -> conexionValida tmc p v) anterior  -- (2) escala >= TMC
      , dentroDelLimite (length tomados)            -- (4) límite K
      , ruta <- dfs (destino v) (destino v : visitados) (Just v) (v : tomados)
      ]
```

El conjunto de visitados no se marca ni se desmarca: cada llamada recursiva recibe una lista nueva (`destino v : visitados`) que solo existe en esa rama, así que cuando una rama fracasa no hay nada que deshacer; la ruta en construcción (`v : tomados`) se acumula en orden inverso y se invierte con `reverse` al llegar al destino. Entender esto nos hizo ver que la inmutabilidad elimina toda una categoría de errores del DFS imperativo.

Funciones de alto orden. Aplicamos `map`, `filter`, `foldr`, `foldl` y `zipWith` donde antes habríamos escrito bucles: `map recortar` limpia los campos de cada línea, `filter ((dia `elem`) . frecuencia)` selecciona los vuelos de un día, `costoTotal` suma los boletos con `foldr`, `minimoPor` recorre las rutas con `foldl` conservando el mejor candidato como acumulador y `esperas` empareja cada vuelo con el siguiente mediante `zipWith`. De `Data.List` usamos `sortOn`, `groupBy`, `nub` e `intercalate`, y `mapM` sobre `Maybe` para que el parseo de la frecuencia falle completo si una abreviatura es desconocida.

Ordenamiento sobre listas y evaluación perezosa. Escribimos `quicksortPor` y `mergesortPor` parametrizados por una función clave (`Ord k =>`): las particiones del *quicksort* son dos *comprehensions* y el *mergesort* divide con `splitAt` y fusiona recursivamente; el desempate se expresa con tuplas como clave, `(costoTotal r, tiempoTotal r)`, porque su instancia `Ord` es lexicográfica. Sobre la pereza aprendimos que `hGetContents` devuelve la cadena antes de leer el disco, por lo que forzamos la lectura con `seq` antes de cerrar el *handle*, y que `take 1 (rutasValidas …)` detiene la exploración en la primera ruta encontrada.

3.2 Prolog

Deconstrucción de listas `[Cabeza|Cola]`. El patrón `(x : xs)` de Haskell es `[X|Xs]` en Prolog, y con él recorremos todas las listas del sistema: `cargar_lineas/4` procesa el archivo línea por línea, `costo_ruta/3` suma los boletos con un acumulador; el patrón de dos elementos `[V1, V2|Resto]` permite a `esperas/2` examinar vuelos consecutivos para calcular la espera en cada escala.

Acumuladores y control de ciclos. El punto de partida natural es la regla clásica `camino(X, Y) :- vuelo(X, Z, ...), camino(Z, Y)`; al estudiarla comprobamos que en un grafo con ciclos puede no terminar. La técnica que investigamos fue el acumulador: `buscar/8` lleva como argumento la lista de ciudades visitadas, que empieza en `[Origen]` y crece con `[Siguiente|Visitados]`, y antes de tomar un vuelo comprueba `\+ memberchk(Siguiente, Visitados)`, una negación por fallo que descarta la rama si la ciudad ya fue visitada. Otros dos acumuladores completan el estado: la llegada del vuelo anterior, necesaria para validar el TMC, y la lista de vuelos tomados, que se invierte con `reverse/2` al final.

```
buscar(Destino, Destino, _, _, _, _, Acum, Acum) :- Acum = [_|_].
buscar(Actual, Destino, Dia, K, Visitados, LlegadaPrev, Acum, Ruta) :-
    Actual \== Destino,
    dentro_del_limite(K, Acum),                                  % (4) límite de escalas
    vuelo(Actual, Siguiente, Salida, Llegada, Costo, Codigo, Dias), % punto de elección
    memberchk(Dia, Dias),                                        % (1) frecuencia
    \+ memberchk(Siguiente, Visitados),                          % (3) control de ciclos
    conexion_valida(LlegadaPrev, Salida),                        % (2) escala >= TMC
    buscar(Siguiente, Destino, Dia, K, [Siguiente|Visitados], Llegada,
           [vuelo(Actual, Siguiente, Salida, Llegada, Costo, Codigo, Dias)|Acum],
           Ruta).
```

Backtracking. `buscar/8` no contiene ningún bucle: la llamada `vuelo(Actual, Siguiente, ...)` deja un punto de elección por cada vuelo que sale de la ciudad actual y, si cualquiera de las metas siguientes falla, el motor retrocede al último punto de elección y prueba la siguiente arista. La búsqueda en profundidad no la programamos: es la estrategia de resolución del propio Prolog. Aprendimos también que el orden de las metas importa para la eficiencia, por lo que colocamos las comprobaciones baratas antes de la llamada recursiva.

Metapredicados. `findall/3` recoge todas las rutas en `todas_las_rutas/5` y devuelve la lista vacía si no hay ninguna; `setof/3` ordena y elimina duplicados pero falla cuando no hay soluciones, y en `ciudades/1` exigió el operador `^` para declarar existenciales las variables ajenas a la plantilla, pues sin él genera una lista por cada combinación de esas variables;

`bagof/3` se comporta como `setof/3` sin ordenar, y con él implementamos `vuelos_del_dia/2`. Dejamos además `ruta_mas_barata_setof/6`, que aprovecha que `setof/3` ya ordena los pares `Costo-Ruta`, para contrastarla con nuestro ordenamiento propio.

Ordenamiento y otros predicados. Implementamos `quicksort/2` (partición con `particion/4` y corte) y `mergesort/2` (división alternada con `partir_mitades/3` y fusión con `fusionar/3`) sobre pares `Clave-Ruta`, con claves compuestas como `c(Costo, Tiempo)` comparadas por el orden estándar de términos (`@<`), lo que da el desempate por el segundo criterio sin código adicional. También estudiamos `keysort/2`, `forall/2`, `maplist/3`, la construcción `( Cond -> Entonces ; Sino )`, el corte `!` y `catch/3`, con el que protegimos el menú de excepciones.

4. Resumen de actividades y responsables

Organizamos el trabajo en tres frentes que coinciden con la estructura del repositorio y con el historial de *commits*. Cada integrante desarrolló su parte desde su propia cuenta de Git con mensajes descriptivos (`feat:`, `fix:`, `docs:`) y revisamos el código de los demás antes de integrarlo en la rama principal.

Integrante	Responsabilidad	Módulos, funciones y predicados
Integrante 1 Maria Rodriguez	Núcleo en Haskell	`Tipos.hs` (tipos algebraicos `Dia`, `Vuelo`, `Ruta`, `Grafo`); `Grafo.hs` (lista de adyacencia); `Consultas.hs`; `Rutas.hs` (`rutasValidas` y la recursión `dfs`); `Optimizacion.hs` (`costoTotal`, `tiempoTotal`, `quicksortPor`, `mergesortPor`, `rutaMasBarata`, `rutaMasRapida`)
Integrante 2 Lynda Nasser	Solución en Prolog	`carga.pl` (base de conocimiento dinámica `vuelo/7`, `tmc/1`); `consultas.pl` (`findall`, `setof`, `bagof`); `rutas.pl` (`ruta_valida/5`, `buscar/8` con *backtracking* y control de ciclos); `optimizacion.pl` (`quicksort/2`, `mergesort/2`, `mas_barata/3`, `mas_rapida/3`)
Integrante 3 Jesus Montoya	Integración, E/S y documentación	`Tiempo.hs` y `tiempo.pl` (conversión `HH:MM` ↔ minutos); `Parser.hs` (parseo de `vuelos.txt`); `Main.hs` y `main.pl` (menús interactivos y validación de entradas); `vuelos.txt`, `README.md`, `.gitignore`, estructura del repositorio y redacción del informe

Las pruebas fueron una actividad compartida: ambas implementaciones se ejecutaron con los mismos casos y cualquier discrepancia entre ellas se trató como un error a investigar.

5. Evidencias de Ejecución por Consola

```
haskell — ghci Main.hs
Ok, 8 modules loaded.
ghci> main
LogiExpress - Motor analítico de rutas aéreas (paradigma funcional)
Cargados 20 vuelos desde ../vuelos.txt

================ LogiExpress (Haskell) ================
Archivo: ../vuelos.txt | Vuelos: 20 | TMC: 45 min
-------------------------------------------------------
1. Cargar archivo de vuelos
2. Configurar TMC (tiempo mínimo de conexión)
3. Vuelos que salen de una ciudad
4. Vuelos que llegan a una ciudad
5. Vuelos que operan un día de la semana
6. Buscar rutas válidas (origen, destino, día, K opcional)
7. Ruta de costo mínimo y ruta de menor tiempo
8. Reporte de rutas ordenadas por costo o por tiempo
9. Listar ciudades
0. Salir
Seleccione una opción: 3
Ciudad de salida: pto ordaz
Vuelos que salen de Pto Ordaz (3):
  Vlo#1    Pto Ordaz -> Maiquetia       07:00-08:00  Bs 5600   [Diario]
  Vlo#6    Pto Ordaz -> Valencia        11:00-12:30  Bs 6700   [Diario]
  Vlo#4    Pto Ordaz -> Maracaibo       13:00-15:00  Bs 9000   [Diario]
```

Figura 1. Haskell: GHCi, `main`, menú principal y vuelos que salen de Pto Ordaz (opción 3).

```
prolog — swipl main.pl
?- iniciar.
LogiExpress - Motor analítico de rutas aéreas (paradigma lógico)
Cargados 20 vuelos desde ../vuelos.txt

================ LogiExpress (Prolog) =================
Archivo: ../vuelos.txt | Vuelos: 20 | TMC: 45 min
-------------------------------------------------------
1. Cargar archivo de vuelos
2. Configurar TMC (tiempo mínimo de conexión)
3. Vuelos que salen de una ciudad
4. Vuelos que llegan a una ciudad
5. Vuelos que operan un día de la semana
6. Buscar rutas válidas (origen, destino, día, K opcional)
7. Ruta de costo mínimo y ruta de menor tiempo
8. Reporte de rutas ordenadas por costo o por tiempo
9. Listar ciudades
0. Salir
Seleccione una opción: 4
Ciudad de llegada: Maiquetia
Vuelos que llegan a Maiquetia (4):
  Vlo#1    Pto Ordaz -> Maiquetia       07:00-08:00  Bs 5600   [Diario]
  Vlo#20   San Antonio -> Maiquetia     15:00-16:30  Bs 8800   [Diario]
  Vlo#5    Maracaibo -> Maiquetia       17:00-18:00  Bs 5500   [Diario]
  Vlo#12   Barquisimeto -> Maiquetia    18:00-19:00  Bs 4500   [L-Mi-V]
```

Figura 2. Prolog: `iniciar.`, menú principal y vuelos que llegan a Maiquetia (opción 4).

```
prolog — swipl -q -g iniciar -t halt main.pl
Seleccione una opción: 6
Ciudad origen: Pto Ordaz
Ciudad destino: Porlamar
Día de la semana (Lunes..Domingo o L,M,Mi,J,V,S,D): Lunes
Máximo de escalas K (Enter = sin límite):
Pto Ordaz -> Porlamar | lunes | TMC 45 min | K = sin límite | 3 ruta(s) válida(s)
Ruta 1: 1 escala(s) | Costo total Bs 10400 | Tiempo total 180 min (3h 00min)
    Vlo#1    Pto Ordaz -> Maiquetia       07:00-08:00  Bs 5600   [Diario]
    Vlo#2    Maiquetia -> Porlamar        09:00-10:00  Bs 4800   [L-M-Mi-J-V]
    Esperas en escalas: 60 min (1h 00min)
Ruta 2: 2 escala(s) | Costo total Bs 13200 | Tiempo total 400 min (6h 40min)
    Vlo#1    Pto Ordaz -> Maiquetia       07:00-08:00  Bs 5600   [Diario]
    Vlo#14   Maiquetia -> Barcelona       11:00-11:50  Bs 4100   [Diario]
    Vlo#15   Barcelona -> Porlamar        13:00-13:40  Bs 3500   [Diario]
    Esperas en escalas: 180 min (3h 00min), 70 min (1h 10min)
Ruta 3: 1 escala(s) | Costo total Bs 11000 | Tiempo total 420 min (7h 00min)
    Vlo#6    Pto Ordaz -> Valencia        11:00-12:30  Bs 6700   [Diario]
    Vlo#9    Valencia -> Porlamar         17:00-18:00  Bs 4300   [L-S-D]
    Esperas en escalas: 270 min (4h 30min)
```

Figura 3. Prolog: rutas válidas Pto Ordaz → Porlamar un lunes con TMC = 45 min y sin límite de escalas (opción 6): tres rutas con su costo, tiempo total y esperas en cada escala.

```
haskell — ./logiexpress
Seleccione una opción: 7
Ciudad origen: Pto Ordaz
Ciudad destino: Porlamar
Día de la semana (Lunes..Domingo o L,M,Mi,J,V,S,D): sabado
Máximo de escalas K (Enter = sin límite):
Pto Ordaz -> Porlamar | Sabado | TMC 45 min | K = sin límite | 2 ruta(s) evaluada(s)
-- Ruta de costo económico mínimo --
Ruta 1: 1 escala(s) | Costo total Bs 11000 | Tiempo total 420 min (7h 00min)
    Vlo#6    Pto Ordaz -> Valencia        11:00-12:30  Bs 6700   [Diario]
    Vlo#9    Valencia -> Porlamar         17:00-18:00  Bs 4300   [L-S-D]
    Esperas en escalas: 270 min (4h 30min)
-- Ruta de menor tiempo total de viaje --
Ruta 1: 2 escala(s) | Costo total Bs 13200 | Tiempo total 400 min (6h 40min)
    Vlo#1    Pto Ordaz -> Maiquetia       07:00-08:00  Bs 5600   [Diario]
    Vlo#14   Maiquetia -> Barcelona       11:00-11:50  Bs 4100   [Diario]
    Vlo#15   Barcelona -> Porlamar        13:00-13:40  Bs 3500   [Diario]
    Esperas en escalas: 180 min (3h 00min), 70 min (1h 10min)
```

Figura 4. Haskell: ruta de costo mínimo y ruta de menor tiempo total para Pto Ordaz → Porlamar un sábado (opción 7); los dos criterios seleccionan rutas distintas.

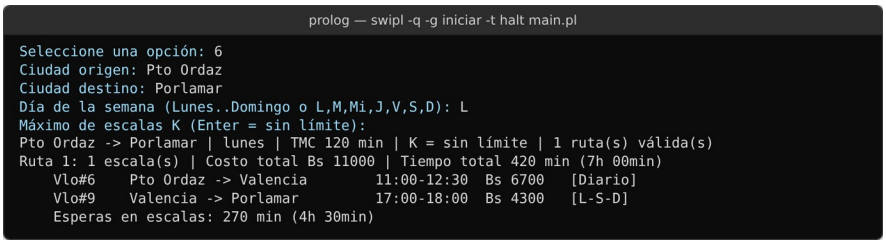


Figura 5. Prolog: la misma búsqueda del lunes con TMC = 120 min; las esperas de 60 y 70 min dejan de ser válidas y solo sobrevive la ruta con espera de 270 min.



Figura 6. Prolog: consulta directa de `ruta_valida/5` en el intérprete; cada `;` fuerza el *backtracking* y obtiene la siguiente ruta hasta agotar las soluciones (`false.`).

6. Conclusiones

El proyecto nos obligó a resolver dos veces el mismo problema, y esa repetición fue lo que nos permitió entender qué aporta cada paradigma.

Haskell: evaluación perezosa e inmutabilidad. Lo más difícil fue el cambio de mentalidad: veníamos de pensar los algoritmos como pasos que modifican variables, y en Haskell no existe `for`, no existe `while` y ninguna variable cambia de valor. Nuestras primeras funciones imitaban bucles con recursión explícita, hasta que comprendimos que casi todos eran en realidad un `map`, un `filter`, un `fold` o una *comprehension*; desde entonces el código se volvió más corto y más fácil de razonar. La inmutabilidad, que parecía una limitación, resultó ser una garantía: el grafo se construye una vez y ninguna búsqueda puede corromperlo, y el control de visitados no necesita deshacer nada porque cada rama trabaja con su propia lista. El sistema de tipos ayudó más de lo que esperábamos (el compilador señaló cada lugar afectado por un cambio en `Vuelo`), y la pereza nos dio una ventaja concreta: `take 1` detiene la búsqueda en la primera ruta. El costo es que el estado hay que enhebrarlo a mano: el TMC y el grafo viajan como argumentos por todas las funciones y el `Estado` del menú se pasa en cada iteración.

Prolog: backtracking y unificación. Ver funcionar `ruta_valida/5` por primera vez fue casi mágico: escribimos qué condiciones debe cumplir una ruta y Prolog encontró todas sin que programáramos ninguna estructura de búsqueda, porque su motor de resolución es en sí mismo una búsqueda en profundidad con retroceso, y la unificación permite consultar los hechos en cualquier dirección. La contrapartida es que esa búsqueda hay que controlarla: en un grafo con ciclos la regla ingenua no termina, y la lista de visitados como acumulador fue la pieza que la hizo correcta. Observamos además que el TMC hace que la hora avance estrictamente a lo largo de una ruta, por lo que la profundidad quedaría acotada incluso sin control de ciclos; aun así, sin la lista de visitados se generarían itinerarios que pasan dos veces por la misma ciudad y se exploraría trabajo inútil. Otras dificultades fueron más sutiles: el corte `!` cambia el significado de un predicado si se coloca mal y los errores suelen manifestarse como un simple `false` que no explica nada, lo que nos llevó a depurar con `trace/0` y consultas parciales. El estado global con `assertz/1` es cómodo pero oculta dependencias: `conexion_valida/2` lee `tmc/1` desde la base de conocimiento, algo que en Haskell es un argumento visible en la firma.

¿Cuál nos pareció mejor para manejar tiempos y grafos? Para la búsqueda en el grafo, Prolog: `buscar/8` es una descripción directa de lo que es una ruta válida y el *backtracking* hace el resto, mientras que en Haskell tuvimos que construir la lista de adyacencia y escribir el DFS. Para el manejo del tiempo y las restricciones, Haskell: los tipos impidieron mezclar minutos con cadenas, `Maybe` nos obligó a tratar las horas inválidas y la separación entre funciones puras e impuras permitió probar todo el motor en GHCi.

Lo que más valoramos es haber descubierto que los dos paradigmas están más cerca de lo que creíamos: en ambos la lista es la estructura central, la solución es recursiva y el control de ciclos usa la misma idea, una lista de visitados que crece en cada paso y nunca se modifica. La diferencia está en quién recorre el espacio de búsqueda: en Prolog, el motor de inferencia mediante puntos de elección y retroceso; en Haskell, una función pura que devuelve de forma perezosa la lista de todas las soluciones. Verificar que ambas implementaciones producen las mismas rutas para todas las consultas posibles nos dio la confianza de que entendimos el problema y no solo dos sintaxis.